Git for Developers

Module 8 — Debugging & History Management

`git bisect`, `git reflog`, `git diff`, and Cleaning Repositories

Module 8 — Agenda

Inspecting Changes

- `git diff` and `git show` — understanding what changed and when
- Binary search debugging with `git bisect`

Recovering and Cleaning

- Reflog: recovering lost commits, branches, and resets
- `git clean` — removing untracked files safely

Part 1 — Inspecting Changes

`git diff` in Depth

```
# Unstaged changes (working directory vs. index)
git diff

# Staged changes (index vs. last commit)
git diff --staged
git diff --cached    # synonym

# Compare working directory directly against last commit
git diff HEAD

# Compare two commits
git diff abc1234 def5678
```

Reading a Diff

```
diff --git a/src/auth.js b/src/auth.js
index a3f7c21..b2e1d40 100644
--- a/src/auth.js
+++ b/src/auth.js
@@ -12,7 +12,8 @@ function validateToken(token) {
   const payload = jwt.decode(token);
-  if (!payload) return null;
+  if (!payload) throw new Error('Invalid token format');
+  if (payload.exp < Date.now() / 1000) throw new Error('Token expired');
   return payload;
 }
```

`git show` — Inspect a Commit

```
# Show last commit: message, author, date, and full diff
git show

# Show a specific commit
git show abc1234

# Show commit metadata only (no diff)
git show --no-patch abc1234
git show --stat abc1234

# Show a specific file as it was in a commit
git show abc1234:src/auth.js
```

`git blame` — Who Wrote This Line?

```
# Annotate a file with commit SHA, author, and date per line
git blame src/auth.js

# Show blame for a specific line range
git blame -L 10,25 src/auth.js

# Ignore whitespace changes
git blame -w src/auth.js

# Follow the line through renames and moves
git blame -C -C -C src/auth.js
```

Part 2 — Bisecting Bugs with `git bisect`

`git bisect` — Binary Search for Bugs

`git bisect` uses binary search to find the exact commit that introduced a bug:

```
# Start a bisect session
git bisect start

# Mark the current commit as bad (bug is present)
git bisect bad

# Mark a known good commit (bug was NOT present here)
git bisect good v2.0.0
```

`git bisect` — Step by Step

```
good: v2.0.0  ─── A ─── B ─── C ─── D ─── bad: HEAD
                                ↑
                        git bisect picks midpoint
```

After 4-5 iterations over 32 commits, Git identifies the exact culprit:

```
a3f7c21 is the first bad commit

commit a3f7c21
Author: Bob Jones <bob@example.com>
Date:   Mon Mar 18 11:42:00 2024
```

Automated `git bisect`

If you can write a test script that exits `0` (pass) or non-zero (fail):

```
git bisect start
git bisect bad HEAD
git bisect good v2.0.0

# Automate: run the test script against every candidate commit
git bisect run ./test-regression.sh
```

```
# test-regression.sh
#!/bin/bash
node -e "const auth = require('./src/auth'); auth.validateToken('test-toke
```

Part 3 — Reflog: Recovering Lost Work

What is the Reflog?

The **reflog** (reference log) records every position `HEAD` and branch tips have been at:

```
# Show the HEAD reflog
git reflog

# Show the reflog for a specific branch
git reflog show main
```

```
a3f7c21 HEAD@{0}: commit: feat: add auth endpoint
b2e1d40 HEAD@{1}: checkout: moving from feature to main
c9a8b12 HEAD@{2}: commit: fix: handle null token
```

Recovering a Dropped Commit

```
# You accidentally ran: git reset --hard HEAD~3
# Find the lost commits in the reflog
git reflog

# Identify the hash before the reset
# e.g., HEAD@{4}: a3f7c21 feat: add user auth

# Recover by creating a branch from that commit
git checkout -b recovery/lost-work a3f7c21

# Or reset back to it
git reset --hard a3f7c21
```

Recovering a Deleted Branch

```
# You deleted a branch: git branch -D feature/user-auth

# Find the last commit on the deleted branch
git reflog | grep 'feature/user-auth'
# or
git reflog --all | grep checkout

# Recreate the branch from its last commit
git checkout -b feature/user-auth a3f7c21
```

Recovering from a Bad Rebase or Reset

```
# Before a risky rebase, note your current position
git log --oneline -3

# Rebase goes wrong — abort first if possible
git rebase --abort

# If you've already completed a bad rebase, use reflog
git reflog
# Find the entry before the rebase started (e.g., "rebase: start rebasing"
git reset --hard HEAD@{5}
```

> The reflog retains entries for **90 days** by default. After that, `git gc`

Part 4 — Cleaning Repositories

`git clean` — Remove Untracked Files

```
# Preview what would be deleted (dry run — always start here!)
git clean -n
git clean --dry-run

# Delete untracked files
git clean -f

# Delete untracked files AND untracked directories
git clean -fd

# Delete ignored files too (build artifacts, etc.)
git clean -fx
```

`git clean` vs. `git restore`

Command	What it removes
`git restore .`	Reverts **tracked** modified files to their last committed state
`git clean -fd`	Removes **untracked** files and directories
Together	Full "reset to clean state"

```
# Nuclear option: restore everything to the last commit
git restore .
git clean -fd
```

Maintaining a Healthy Repository

```
# Run garbage collection and pack loose objects
git gc

# Check repository integrity
git fsck

# Prune unreachable objects (run after git gc)
git prune

# Expire old reflog entries and run gc
git reflog expire --expire=90.days.ago --all
git gc --prune=now
```

Summary — Debugging & History

Command	What it does
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git show <hash>`	Show commit details and diff
`git blame <file>`	Annotate file lines with commit info
`git bisect start/good/bad`	Binary search for a bug-introducing commit
`git bisect run <script>`	Automated bisect